

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №9**

**«Паралельність, багатопроцесність та оптимізація: профілювання
статистики студентів»**

з курсу «Професійний Python»
Варіант №1 — «Система обліку студентів»

Виконав: Лис Артур

Група: ФЕП-31

Перевірив(ла): _____

Дата: 08.09.2026

Львів — 2026


```
def make_students(count: int, seed: int = 42) -> list[Student]:
    """`count` students spread over 20 groups with grades in 40.0..100.0 (one decimal)."""
    rng = random.Random(seed)
    return [
        Student(rng.choice(FIRST_NAMES), rng.choice(LAST_NAMES), rng.choice(GROUPS),
round(rng.uniform(40, 100), 1))
        for _ in range(count)
    ]
```

Тестовий набір: 200 000 студентів 20 груп оцінки 40..100 (seed 42)

 згенеровано за 0.58 s; перший: Student(first_name='Остан', last_name='Гнатишин', group='ФЕП-32',
 average_grade=54.7)
 Python: 3.13.0, CPU: 12 логічних ядер

200 000 валідних об'єктів Student (кожен проходить перевірку групи та балу з ЛР1), 20 груп, оцінки 40.0–100.0 з одним знаком після коми, фіксований seed — набір відтворюваний між запусками.

5. Baseline і перевірка правильності

```
def compute_stats(students: Sequence[Student]) -> Stats:
    if not students:
        raise ValueError("порожній набір студентів")
    total, low, high = 0.0, inf, -inf
    sums: dict[str, float] = {}
    counts: dict[str, int] = {}
    for student in students:
        grade = student.average_grade
        total += grade
        low = min(low, grade)
        high = max(high, grade)
        sums[student.group] = sums.get(student.group, 0.0) + grade
        counts[student.group] = counts.get(student.group, 0) + 1
    n = len(students)
    mean = total / n
    squares = 0.0
    for student in students:
        squares += (student.average_grade - mean) ** 2
    group_means = {code: sums[code] / counts[code] for code in sorted(sums)}
    ranking = sorted(range(n), key=lambda i: students[i].average_grade, reverse=True)
    return Stats(n, mean, low, high, sqrt(squares / n), group_means, ranking)
```

Baseline (Python loops): результат і правильність

 count=200000 mean=69.9794 min=40.0 max=100.0 std=17.3010
 group_means (перші 3): {'ФЕП-21': 70.139, 'ФЕП-22': 70.13, 'ФЕП-23': 69.917}
 top-3 ranking: [('Петришин Олег', 100.0), ('Гаврилюк Богдан', 100.0), ('Бойко Остан', 100.0)]
 контроль через statistics/NumPy: mean=69.9794 std=17.3010 -> збігається: True

stats_close порівнює дві структури Stats із допуском на похибку float, а рейтинг — точно; саме нею далі перевіряється кожна оптимізована версія. Контрольні значення mean і std, обчислені незалежно NumPy, збігаються з baseline.

6. timeit: кілька повторів

```
def measure(label: str, func: Callable[[], object], *, repeats: int = 5, number: int = 1) -> Timing:
    """timeit.repeat: `repeats` independent runs of `number` calls; the best run is the least disturbed
    one."""
    runs = tuple(t / number for t in timeit.repeat(func, number=number, repeat=repeats))
    return Timing(label, min(runs), sum(runs) / len(runs), runs)
```

timeit baseline: 5 повторів

```
-----
runs: ['0.1356', '0.1341', '0.1403', '0.1386', '0.1345'] best=0.1341 s mean=0.1366 s
```

П'ять незалежних прогонів; за оцінку береться найкращий (найменш «зашумлений» іншими процесами), mean показує розкид. Одиначні виміри perf_counter для такого коду коливаються на 5–10 %.

7. cProfile: де саме витрачається час

cProfile baseline (top за tottime)

```
-----
List reduced from 10 to 8 due to restriction <8>
ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1   0.166    0.166    0.347    0.347 baseline.py:23(compute_stats)
      2   0.065    0.032    0.080    0.040 {built-in method builtins.sorted}
400000   0.058    0.000    0.058    0.000 {method 'get' of 'dict' objects}
200000   0.022    0.000    0.022    0.000 {built-in method builtins.max}
200000   0.022    0.000    0.022    0.000 {built-in method builtins.min}
200000   0.016    0.000    0.016    0.000 baseline.py:42(<lambda>)
      1   0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
      1   0.000    0.000    0.000    0.000 <string>:2(__init__)
```

ncalls — кількість викликів, tottime — власний час функції без викликаних нею, cumtime — разом із ними. Тіло compute_stats (0.16 с tottime) — інтерпретація байткоду циклу над 200 000 об'єктів; sorted для рейтингу — 0.063 с; 400 000 викликів dict.get для сум і лічильників груп — 0.057 с; min/max як функції — по 0.021 с. Bottleneck — не одна «важка» функція, а сама кількість дрібних Python-операцій на кожен елемент. Це підказує два напрямки: прибрати Python-цикл (NumPy) або розділити його між кількома інтерпретаторами (процеси); потоки не допоможуть, бо байткод виконує один GIL.

8. tracemalloc: пам'ять

```
def peak_memory(func: Callable[[], T]) -> tuple[T, int]:
    """Run under tracemalloc; return (result, peak bytes allocated during the call)."""
    tracemalloc.start()
    try:
        result = func()
        _, peak = tracemalloc.get_traced_memory()
    finally:
        tracemalloc.stop()
    return result, peak
```

tracemalloc: peak memory

```
-----
baseline compute_stats: peak 10.7 MB
```

```
to_arrays (list -> ndarray): peak 3.1 MB; масиви: 3.1 MB
compute_stats_numpy: peak 9.1 MB
list[Student] (оцінка, без рядків): 13.8 MB проти ndarray grades 1.5 MB
висновок: операція CPU-bound – жодного I/O, час іде на байткод циклу та sorted
```

Пік baseline — 10.7 MB: список рейтингу з 200 000 int та словники. Список об'єктів Student займає близько 14 MB без урахування рядків, тоді як масив оцінок NumPy — 1.5 MB (8 байтів на float64 без заголовка об'єкта). Це не bottleneck за пам'яттю, але пояснює, чому векторизація ще й економніша.

9. Потoki: ThreadPoolExecutor, threading.Thread, Lock

Усі паралельні версії будуються з одного «мергованого» часткового результату Partial: суми, суми квадратів, min/max, суми та лічильники за групами й відсортований рейтинг шматка. Стандартне відхилення рахується з суми квадратів, рейтинг — злиттям відсортованих прогонів.

```
@dataclass(slots=True)
class Partial:
    count: int = 0
    total: float = 0.0
    squares: float = 0.0
    minimum: float = inf
    maximum: float = -inf
    group_sum: dict[GroupKey, float] = field(default_factory=dict)
    group_count: dict[GroupKey, int] = field(default_factory=dict)
    neg_grades: list[float] = field(default_factory=list) # -grade, ascending (= grades descending)
    indices: list[int] = field(default_factory=list)      # global student index in the same order

def stats_threads(students: Sequence[Student], workers: int = 4) -> Stats:
    """ThreadPoolExecutor: the chunks are processed by threads that share one interpreter (and one
    GIL)."""
    pieces = split(students, workers)
    with ThreadPoolExecutor(max_workers=workers) as pool:
        return merge(pool.map(partial_stats, [c for _, c in pieces], [o for o, _ in pieces]))

def stats_raw_threads(students: Sequence[Student], workers: int = 4) -> Stats:
    """The same with threading.Thread by hand: start/join and a Lock around the shared result list."""
    results: list[Partial] = []
    lock = threading.Lock()

    def work(chunk: Sequence[Student], offset: int) -> None:
        part = partial_stats(chunk, offset) # computed outside the lock: long, no shared state
        with lock:                          # critical section: the shared list
            results.append(part)

    threads = [threading.Thread(target=work, args=(chunk, offset)) for offset, chunk in split(students,
workers)]
    for thread in threads:
        thread.start()
    for thread in threads:
        thread.join()
    return merge(results)
```

9.1. Race condition і виправлення через Lock

```
class Counter:
    def __init__(self) -> None:
        self.value = 0
        self.lock = threading.Lock()

    def increment_unsafe(self) -> None:
        current = self.value          # read
        self.value = self._plus_one(current) # write; the call in between is where the interpreter may
switch                                # threads, so another thread's write gets overwritten
        (lost update)

    @staticmethod
    def _plus_one(value: int) -> int:
        return value + 1

    def increment_safe(self) -> None:
        with self.lock:
            self.value += 1
```

Race condition: 4 потоки × 200 000 інкрементів (switch interval 5 ms за замовчуванням, потім 1 μs)

```
-----
switch interval 5 ms (default) без Lock: 718079 із 800000 (втрачено 81921); з Lock: 800000
switch interval 1 μs          без Lock: 416796 із 800000 (втрачено 383204); з Lock: 800000
час: без Lock 0.115 s, з Lock 0.275 s – синхронізація коштує 2.4×
```

Без Lock два потоки читають одне значення і обидва записують «+1» — одне оновлення губиться. При типовому інтервалі перемикання 5 мс втрачено 7 % інкрементів, при 1 мкс — 47 %. Lock робить критичну секцію атомарною, але коштує 2.4× часу — синхронізація не безкоштовна, тому у stats_raw_threads під замком лише append результату, а не сам підрахунок.

10. Процеси: ProcessPoolExecutor

```
def stats_processes(students: Sequence[Student], workers: int = 4) -> Stats:
    """ProcessPoolExecutor, naive: Student objects are pickled to the workers, a new pool every
call."""
    pieces = split(students, workers)
    with ProcessPoolExecutor(max_workers=workers) as pool:
        return merge(pool.map(partial_stats, [c for _, c in pieces], [o for o, _ in pieces]))
```

```
def stats_processes_columns(columns: Columns, workers: int = 4, pool: Executor | None = None) -> Stats:
    """ProcessPoolExecutor, tuned: primitive columns cross the boundary, and an existing (warm) pool is
reused."""
    n = len(columns.grades)
    size = max(1, -(n // workers))
    offsets = list(range(0, n, size))
    grades = [columns.grades[o:o + size] for o in offsets]
    ids = [columns.group_ids[o:o + size] for o in offsets]
    if pool is None:
        with ProcessPoolExecutor(max_workers=workers) as own:
            return merge(own.map(partial_stats_columns, grades, ids, offsets), columns.codes)
    return merge(pool.map(partial_stats_columns, grades, ids, offsets), columns.codes)
```

Наївна версія пересилає 200 000 об'єктів Student у workers через pickle і щоразу створює пул (на Windows процеси стартують через spawn — новий інтерпретатор і повторний імпорт). Налаштована версія пересилає лише два списки примітивів (to_columns, 0.02 с) і використовує вже запущений «теплий» пул.

11. NumPy: векторизація

```
def to_arrays(students: Sequence[Student]) -> Arrays:
    """One-time conversion (this is the cost NumPy has to pay back)."""
    codes = sorted({s.group for s in students})
    index = {code: i for i, code in enumerate(codes)}
    grades = np.fromiter((s.average_grade for s in students), dtype=np.float64, count=len(students))
    group_ids = np.fromiter((index[s.group] for s in students), dtype=np.int64, count=len(students))
    return Arrays(grades, group_ids, codes)


def compute_stats_numpy(arrays: Arrays) -> Stats:
    grades, ids = arrays.grades, arrays.group_ids
    if grades.size == 0:
        raise ValueError("порожній набір студентів")
    sums = np.bincount(ids, weights=grades, minlength=len(arrays.codes))
    counts = np.bincount(ids, minlength=len(arrays.codes))
    group_means = {code: float(sums[i] / counts[i]) for i, code in enumerate(arrays.codes) if counts[i]}
    ranking = np.argsort(-grades, kind="stable") # stable: ties keep input order, like
    sorted(reverse=True)
    return Stats(int(grades.size), float(grades.mean()), float(grades.min()), float(grades.max()),
                 float(grades.std()), group_means, ranking.tolist())
```

Жодного циклу на рівні Python: середнє, min, max, std — методи масиву; середні за групами — два bincount; рейтинг — argsort зі stable, щоб рівні бали зберігали вхідний порядок так само, як sorted(reverse=True). Конвертація списку об'єктів у масиви вимірюється окремо, бо в реальному проєкті її можна зробити один раз.

12. Кешування

```
class GroupStatisticsService:
    def __init__(self, students: Sequence[Student], maxsize: int = 64) -> None:
        self._students = list(students)
        self._version = 0
        self._cached = lru_cache(maxsize=maxsize)(self._compute)

    def _compute(self, code: str, version: int) -> GroupStats: # noqa: ARG002 - version is part of the key
        total, count, low, high = 0.0, 0, inf, -inf
        for student in self._students:
            if student.group == code:
                grade = student.average_grade
                total += grade
                count += 1
                low = min(low, grade)
                high = max(high, grade)
        if count == 0:
```

```

        raise KeyError(code)
    return GroupStats(code, count, total / count, low, high)

def group_statistics(self, code: str) -> GroupStats:
    return self._cached(code, self._version)

def add(self, student: Student) -> None:
    """Data changed -> new version -> every old key misses (implicit invalidation)."""
    self._students.append(student)
    self._version += 1

```

Кешування статистики групи (lru_cache, ключ = код + версія даних)

```

cold (miss): 8.43 ms; warm (hit): 0.2 μs; speedup 42 174x
після 5 запитів (3 різні групи): hits, misses, size = (8, 3, 3)
після add() (версія змінилась) запит ФЕР-31 знову miss: (8, 4, 4)
після invalidate(): (0, 0, 0)

```

Ключ кешу — код групи плюс версія даних: add() збільшує версію, і всі старі відповіді стають miss без явного очищення; invalidate() — явна форма. Cold-запит проходить 200 000 об'єктів за 8 мс, warm — 0.2 мкс: кешування дає найбільший speedup з усіх методів, але лише для повторних однакових запитів.

13. Результати експерименту

Правильність усіх реалізацій відносно baseline

```

threads x4           збігається: True
raw threads + Lock x4 збігається: True
processes x4         збігається: True
numpy                збігається: True

```

Benchmark (5 повторів, best/mean), speedup = baseline.best / best

```

to_columns (об'єкти -> списки float/int): 0.0197 s

```

метод	best, s	mean, s	speedup
Python loops (baseline)	0.1341	0.1366	1.00x
ThreadPoolExecutor x1	0.2439	0.2544	0.55x
ThreadPoolExecutor x2	0.2261	0.2347	0.59x
ThreadPoolExecutor x4	0.2370	0.2459	0.57x
ThreadPoolExecutor x8	0.2033	0.2135	0.66x
ProcessPool x4, об'єкти, новий пул	1.0609	1.1544	0.13x
ProcessPool x4, колонки, новий пул	0.6773	0.6815	0.20x
ProcessPool x1, колонки, теплий пул	0.2673	0.2763	0.50x
ProcessPool x2, колонки, теплий пул	0.1535	0.1631	0.87x
ProcessPool x4, колонки, теплий пул	0.1131	0.1220	1.19x
ProcessPool x8, колонки, теплий пул	0.1011	0.1038	1.33x
Numpy (з конвертацією)	0.0530	0.0549	2.53x
Numpy (масиви готові)	0.0211	0.0221	6.34x

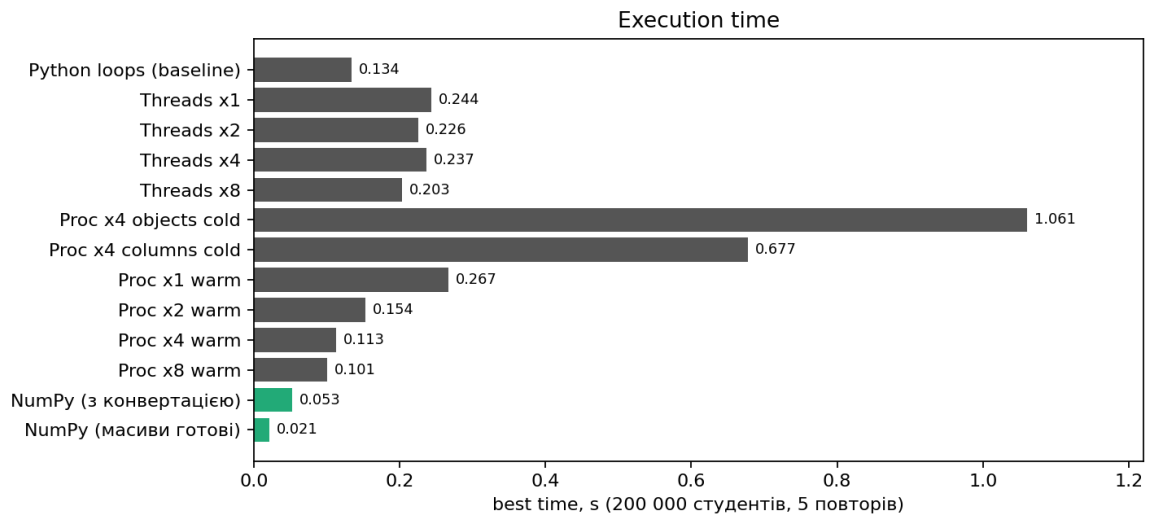


Рис. 1. Час виконання (best із 5 повторів), 200 000 студентів

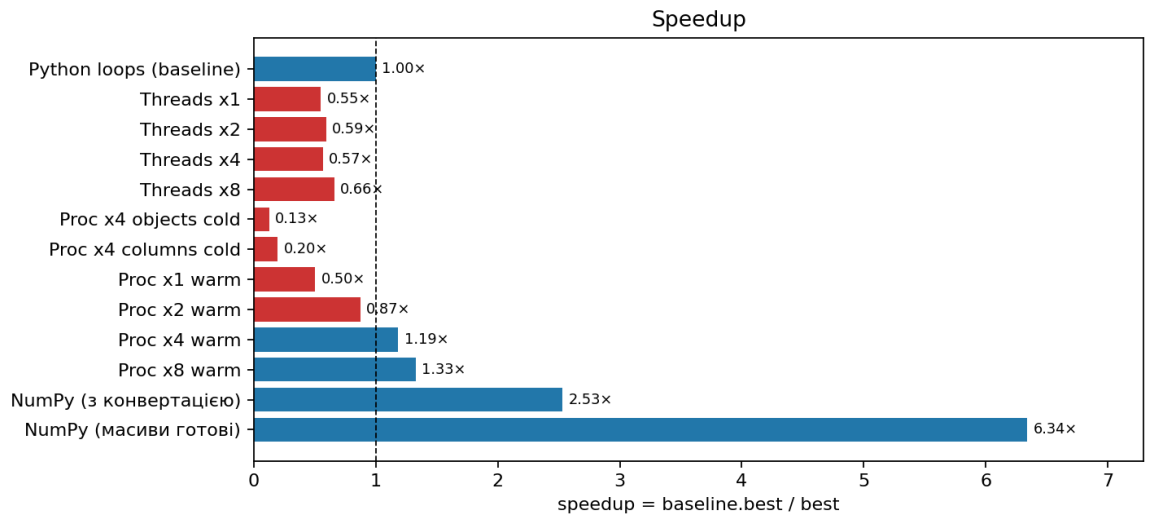


Рис. 2. Speedup відносно baseline; червоні — повільніші за baseline

Метод	best, s	speedup	Пояснення
Python loops	0.134	1.00×	baseline
ThreadPoolExecutor ×1...×8	0.20–0.24	0.55–0.66×	GIL: байткод виконує один потік; додається розбиття, злиття та створення потоків
ProcessPool ×4, об'єкти, новий пул	1.061	0.13×	pickle 200 000 об'єктів + spawn 4 інтерпретаторів

ProcessPool ×4, колонки, новий пул	0.677	0.20×	payload дешевий, але spawn лишається
ProcessPool ×1/×2/×4/×8, теплий пул	0.267/0.154/0.113/0.101	0.50/0.87/1.19/1.33×	справжній паралелізм; ×1 показує ціну серіалізації (~0.13 с)
NumPy з конвертацією	0.053	2.53×	0.03 с — перетворення об'єктів у масиви
NumPy, масиви готові	0.021	6.34×	цикл у C, без об'єктів Python

13.1. Кількість workers і розмір даних

Різні розміри даних: baseline / NumPy (з конвертацією) / теплий ProcessPool x4 (колонки)

size	baseline	numpy	proc x4
10000	0.0055	0.0024	0.0057
50000	0.0317	0.0141	0.0262
200000	0.1356	0.0571	0.1270
500000	0.3545	0.1498	0.3399

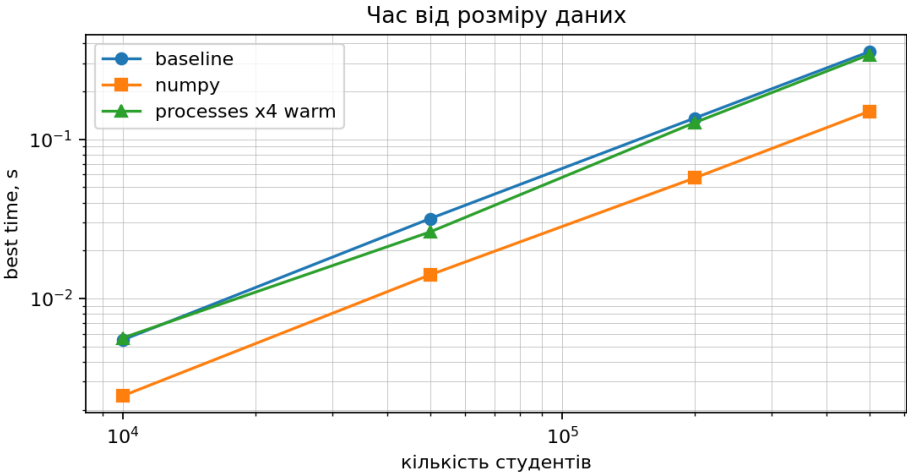


Рис. 3. Час від розміру даних (лог-шкали): NumPy стабільно ~2.4× швидше, теплий пул виграв лише від ~50k

Для потоків кількість workers нічого не змінює — очікувано для CPU-bound коду під GIL. Для процесів вигреш росте до 4 workers (1.19×) і майже зупиняється на 8 (1.33×): 12 логічних ядер, але кожен worker мусить отримати свій шматок через pickle і повернути рейтинг назад, а злиття виконується в одному процесі — за законом Амдала послідовна частина обмежує прискорення. Оптимум — 4–8 workers. На 10 000 записів процеси вже програють baseline: накладні витрати перевищують роботу.

14. Аналіз експерименту

- Bottleneck — CPU-bound цикл інтерпретатора над 200 000 об'єктів; профіль показав, що жодна окрема функція не домінує, отже потрібна зміна моделі виконання, а не мікрооптимізація однієї функції.
- Потоки для цієї задачі шкідливі (0.55–0.66×): GIL пускає до байткоду один потік, а розбиття і злиття додають роботи. Вони мали б сенс для I/O-bound операцій (читання файлів ЛР3/ЛР5, HTTP-запити ЛР8).
- Процеси дають справжній паралелізм, але лише якщо (а) пул уже запущено, (б) дані передаються компактно; інакше serialization і spawn коштують у 5–8 разів більше, ніж уся робота.
- NumPy — найкращий метод для розрахунку статистики: 6.3× на готових масивах і 2.5× навіть з конвертацією; кешування виграє лише повторні запити (42 000×), тому воно доповнює NumPy, а не замінює.
- Кожна реалізація перевірена stats_close проти baseline — оптимізація, що змінює результат, не рахується.

15. Тести, перевірка типів і Git

```
$ pytest tests/unit/test_perf.py -v
tests/unit/test_perf.py::test_baseline_values PASSED
tests/unit/test_perf.py::test_baseline_rejects_empty PASSED
tests/unit/test_perf.py::test_optimised_versions_match_baseline[threads] PASSED
tests/unit/test_perf.py::test_optimised_versions_match_baseline[raw-threads] PASSED
tests/unit/test_perf.py::test_optimised_versions_match_baseline[numpy] PASSED
tests/unit/test_perf.py::test_process_pool_matches_baseline PASSED
tests/unit/test_perf.py::test_columns_variant_matches_baseline_in_process PASSED
tests/unit/test_perf.py::test_lost_updates_appear_with_a_tiny_switch_interval PASSED
tests/unit/test_perf.py::test_split_covers_everything_and_merge_is_exact PASSED
tests/unit/test_perf.py::test_cache_hits_misses_and_invalidation PASSED
tests/unit/test_perf.py::test_lock_prevents_lost_updates PASSED
tests/unit/test_perf.py::test_measuring_tools PASSED
===== 12 passed in 1.46s =====
```

```
$ pytest -q
225 passed in 14.37s
```

```
$ mypy src
Success: no issues found in 84 source files
```

```
$ git log --oneline -4
78bae4d lab9: correctness tests for every implementation, README section, version 0.9.0, mypy
python_version 3.12
93c3cb7 lab9: threads/processes with mergeable partials, warm pool + column payload, Lock and race
demo, lru_cache with data version
08f411f lab9: perf package – synthetic dataset, loop baseline, NumPy version,
timeit/cProfile/tracemalloc helpers
```

c75419d lab8: API tests through ASGITransport and MockTransport, pytest-asyncio, README section, version 0.8.0

```
@pytest.mark.parametrize("implementation", [
    partial(stats_threads, workers=3), partial(stats_raw_threads, workers=3),
    compute_stats_numpy_from_students,
], ids=["threads", "raw-threads", "numpy"])
def test_optimised_versions_match_baseline(students: list[Student], implementation: object) -> None:
    assert stats_close(compute_stats(students), implementation(students)) # type: ignore[operator]

def test_lost_updates_appear_with_a_tiny_switch_interval() -> None:
    assert race(50_000, threads=4, use_lock=True, switch_interval=1e-6) == 200_000
    assert race(50_000, threads=4, use_lock=False, switch_interval=1e-6) < 200_000

def test_split_covers_everything_and_merge_is_exact(students: list[Student]) -> None:
    pieces = split(students, 7)
    assert sum(len(chunk) for _, chunk in pieces) == len(students)
    assert [offset for offset, _ in pieces] == sorted(offset for offset, _ in pieces)
    merged = merge(partial_stats(chunk, offset) for offset, chunk in pieces)
    assert stats_close(compute_stats(students), merged)

def test_cache_hits_misses_and_invalidation(students: list[Student]) -> None:
    service = GroupStatisticsService(students)
    first = service.group_statistics("ФЕП-31")
    assert service.group_statistics("ФЕП-31") == first
    assert service.info() == (1, 1, 1)
    service.add(Student("Новий", "Студент", "ФЕП-31", 100.0))
    assert service.group_statistics("ФЕП-31").maximum == 100.0 # new version -> recomputed
    assert service.info()[1] == 2
    service.invalidate()
    assert service.info() == (0, 0, 0)
    with pytest.raises(KeyError):
        service.group_statistics("HEMA-00")

def test_lock_prevents_lost_updates() -> None:
    assert race(20_000, threads=4, use_lock=True) == 80_000
    assert race(20_000, threads=4, use_lock=False) <= 80_000
```

16. Відповіді на контрольні запитання

1. Чим concurrency відрізняється від parallelism?

Concurrency — кілька задач просуваються по чергову (asyncio, потоки під GIL); parallelism — справді одночасно на різних ядрах (процеси, NumPy у C).

2. Чим process відрізняється від thread?

Процес має власну пам'ять та інтерпретатор; потоки ділять пам'ять одного процесу і в CPython — один GIL.

3. Що таке GIL і яка його роль у CPython?

Глобальне блокування інтерпретатора: байткод у кожен момент виконує один потік; спрощує керування пам'яттю, але блокує паралелізм CPU-bound потоків.

4. CPU-bound та I/O-bound задачі — приклади?

CPU-bound: статистика 200 000 записів, сортування; I/O-bound: читання файлу, HTTP-запит — процесор чекає.

5. Коли доцільні threading, а коли multiprocessing?

Потоки — I/O-bound (під час очікування GIL звільняється); процеси — CPU-bound, якщо дані дешево передати.

6. Для чого start() і join()?

start() запускає потік, join() чекає його завершення, щоб зібрати результат.

7. Що таке race condition, critical section, Lock?

Втрата коректності через одночасний доступ до спільного стану; ділянка коду, що працює з ним; замок, який пускає в неї один потік.

8. Чи завжди synchronization покращує performance?

Ні, вона гарантує коректність і коштує часу (2.4× у демонстрації); чим менша критична секція, тим менший overhead.

9. Для чого executor.map() і що таке worker?

Розподіляє елементи між worker-ами (потоками чи процесами пулу) і повертає результати в порядку входу.

10. Чи завжди більше workers — швидше?

Ні: за 4–8 workers виграш зупинився — послідовна частина (серіалізація, злиття) та кількість ядер обмежують.

11. Що таке multiprocessing overhead і serialization?

Запуск інтерпретаторів (spawn) і pickle даних туди й результатів назад; для 200 000 об'єктів це 1 с проти 0.13 с роботи.

12. Для чого timeit, perf_counter і кілька повторів?

Точний монотонний таймер; timeit прибирає системний шум серією повторів — беремо best, дивимося на розкид.

13. Що показують ncalls, tottime, cumtime?

Кількість викликів; власний час функції; час разом із викликаними функціями.

14. Для чого `tracemalloc` і що таке `peak memory`?

Відстежує виділення пам'яті Python-об'єктами; `peak` — максимум у ході операції (10.7 MB для `baseline`).

15. Чому NumPy швидший за Python loops?

Векторизація: один виклик виконує цикл у C над щільним масивом без створення об'єктів і диспетчеризації байткоду.

16. `@cache`, `@lru_cache`, `cache hit/miss`, `invalidation`?

Мемоізація результату за аргументами; `hit` — відповідь з кешу, `miss` — обчислення; інвалідація — скидання після зміни даних (тут — версія у ключі).

17. Коли кешування небезпечне?

Коли дані змінюються, а кеш про це не знає — відповіді стають застарілими; або коли ключі нескінченно різні й кеш з'їдає пам'ять.

18. Як обчислюється `speedup` і що означає `speedup < 1`?

$T_{\text{baseline}} / T_{\text{optimised}}$; менше 1 — «оптимізація» повільніша за `baseline` (потоки тут — $0.55\times$).

19. Чому оптимізацію треба починати з `profiling` і перевіряти правильність?

Без профілю прискорюють не те, що гальмує; без перевірки — швидко отримують неправильну відповідь.

17. Висновок

Обчислення статистики 200 000 студентів профільовано і виявлено CPU-bound bottleneck у циклі інтерпретатора. З чотирьох підходів найефективніший — NumPy ($6.3\times$ на готових масивах, $2.5\times$ з конвертацією), процеси дають $1.33\times$ лише з теплим пулом і компактним `payload`, потоки уповільнюють (GIL), а кешування прискорює повторні запити на чотири порядки. Усі версії відтворюють результат `baseline` точно; 225 тестів проєкту проходять, туру чистий. Головний висновок роботи: цифри залежать від накладних витрат не менше, ніж від алгоритму, тому кожен метод треба міряти на власних даних.